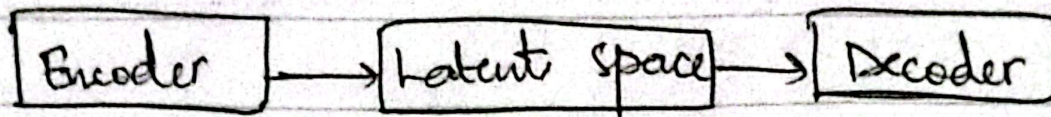


Encoder Decoder Architecture



Machine Translation :-

1. Encoder (Input processing) :-

- The encoder reads the input sentence & compresses it into latent representation.

Idea:-

↳ Converts high dimensional sequence $\rightarrow$ low dimensional vector (latent space).

↳ This vector contains the meaning of whole sentence.

Whole sentence :-

"जो कब. सोल (म)

Encoder processes each word step by step.

$w_1 \rightarrow w_2 \rightarrow w_3 \rightarrow w_4$

Final output:-

Latent vector (context)

Latent Space :-

- A fixed length vector
 - Acts as a summary of the entire input sentence.
 - Contains semantic meanings (not just words).
- This is the "memory" passed to the decoder.

Decoder (Output generation) :-

The decoder generates the output sentence one by one.

- Starts with START TOKEN.
- generates next word.
- Feeds predicted word as next input.
- Stops at the END TOKEN.

Teacher Forcing :-

↳ Diff. b/w training & testing.

Training Phase (Teacher Forcing) :-

Instead of using predicted outputs:-
We use the actual correct word.

Example :-

- True output :- "I go to school".
 - Model predicted : "I am to school"
- ↳ still next input = correct word ("go"), NOT predicted word.

Why:-

- ↳ Makes training faster
- ↳ Stabilizes learning
- ↳ Prevents error accumulation

Testing Phase:-

- ↳ No Ground Truth available.
 - ↳ Model uses its own predictions.
- Each predicted word becomes the next input.

Architecture:-

Input sentence $\rightarrow$ Encoder $\rightarrow$ Latent vector $\rightarrow$
Decoder $\rightarrow$ Output Sentence.

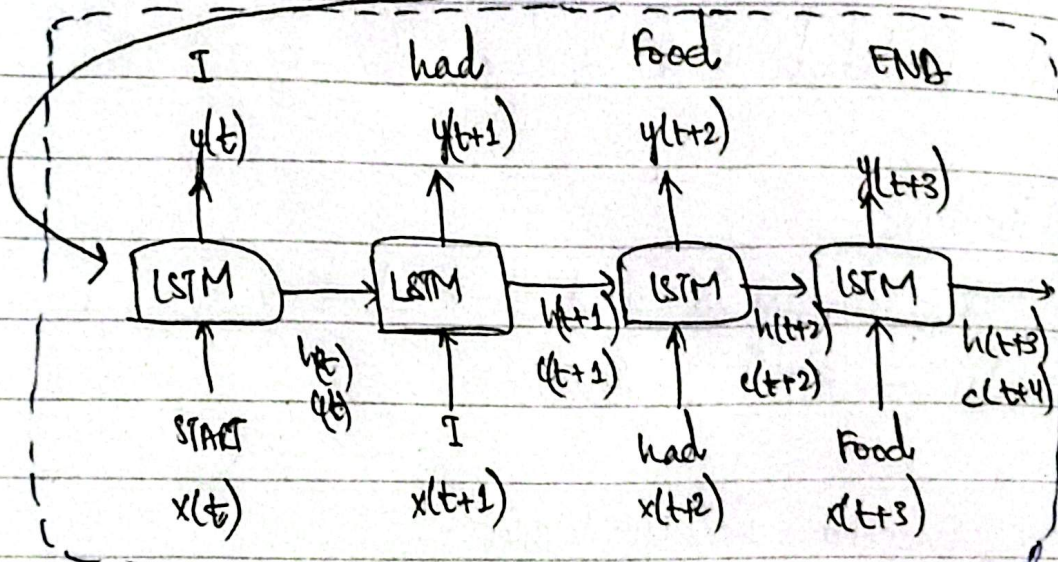
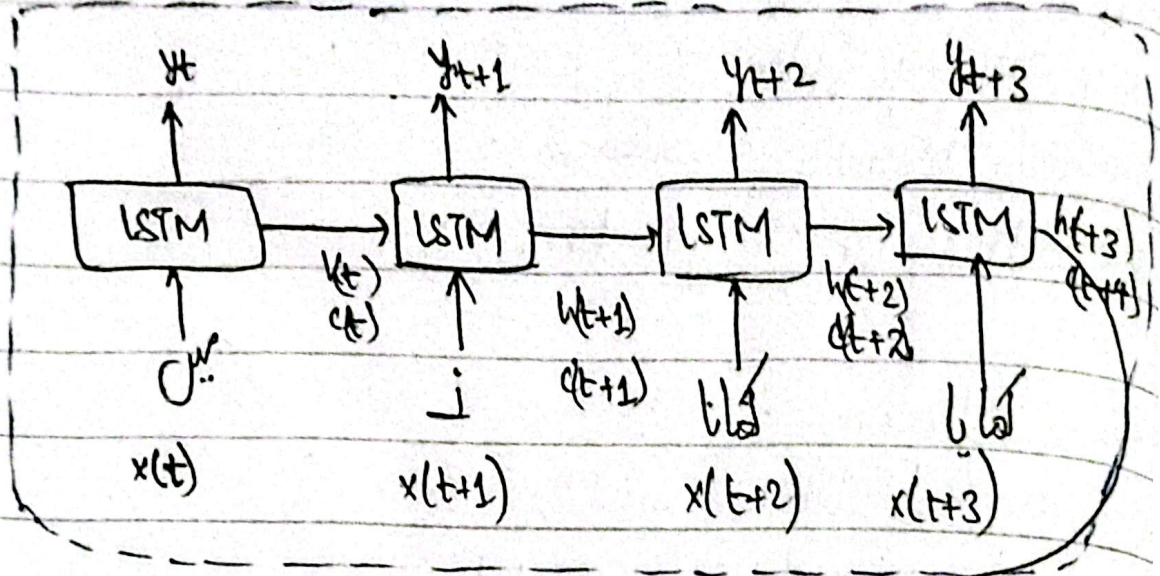
Why Encoder - Decoder Works:-

- Handles variable-length input & output.
- Learns sequence-to-sequence mapping.
- Captures meaning of full sentence.

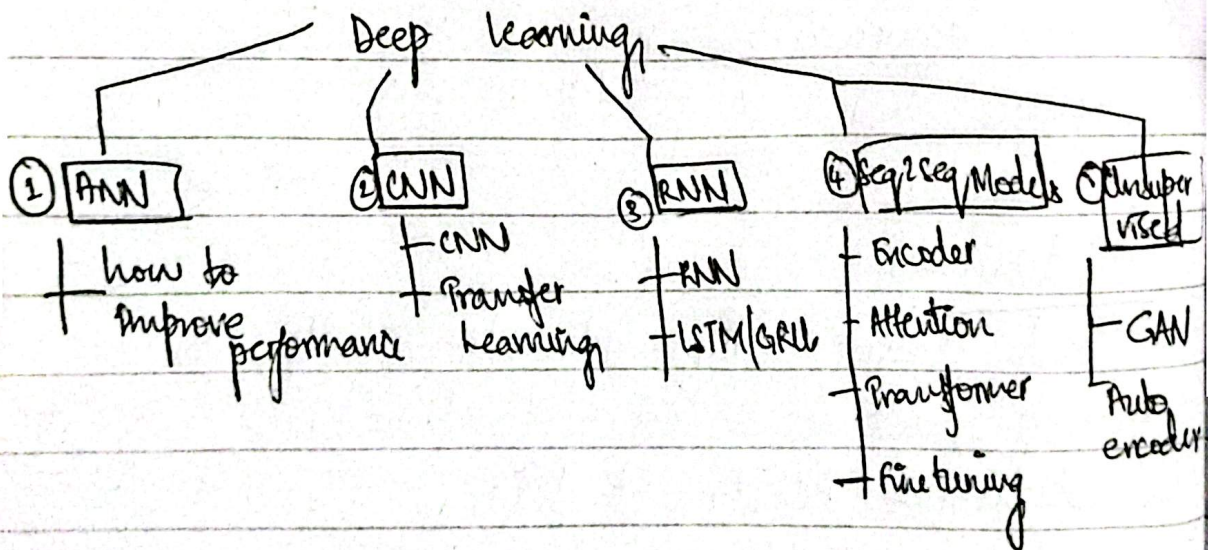
Limitations:-

- ↳ Fixed latent vector = information bottleneck
- ↳ Long sentences = loss of information
- ↳ Solution = Attention Mechanisms.

Encoder



Decoder



seq² seq → RNN → sequential → sequence

sequence branches into: language/sentence, time series, bioinformatics

Types of RNN

many-to-one
sentiment
analysis

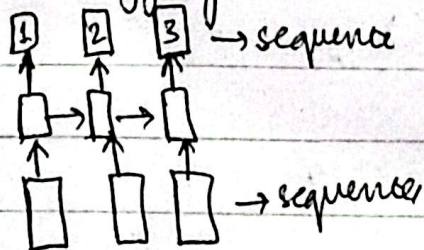
text → sentiment

one-to-many
image caption generation

input → image caption (text) → output

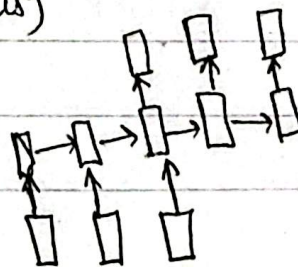
many-to-many (synchronous)

- ↳ NER (Named Entity Recognition)
- ↳ POS tagging



many-to-many (asynchronous)

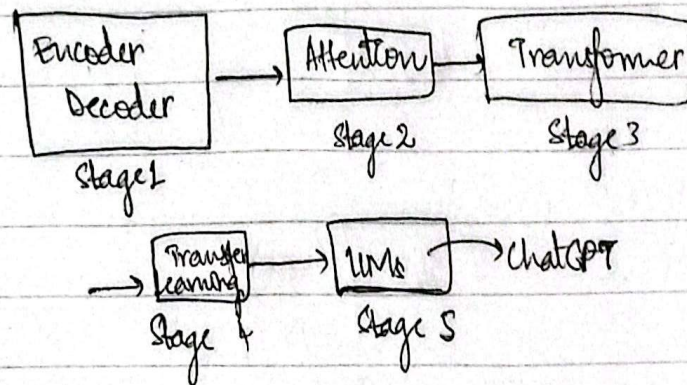
- ↳ machine translation



Seq2Seq Tasks :-

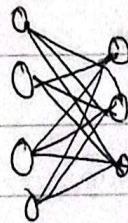
- ↳ machine translation
- ↳ text summarization
- ↳ Question Answer
- ↳ Chatbot / conversational AI
- ↳ Speech to Text

History of Seq2Seq Models :-



Seq2Seq Data

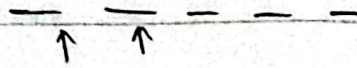
1) Tabular → [ANN]



2) Image Data → CNN

3) Sequential → textual

→ time series



RNN

LSTMs

Gell

4) Seq²Seq data

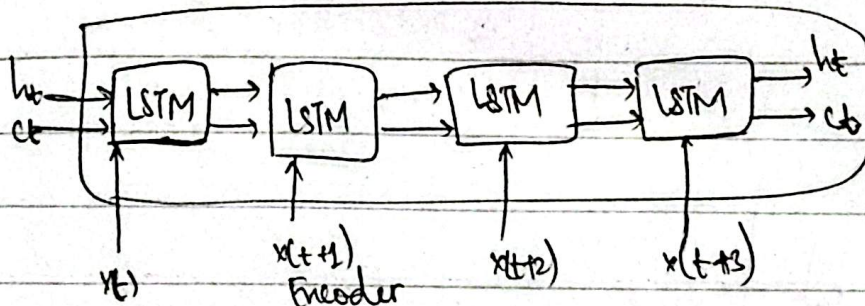
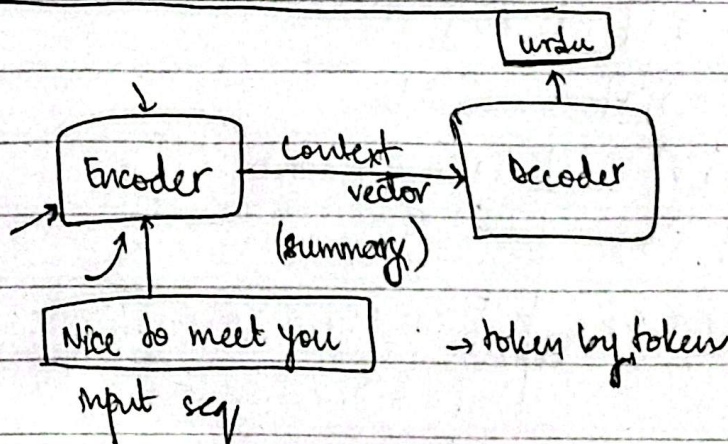
→ machine → language translation

Challenges :-

input → english → variable lengths

output → urdu → variable lengths

Encoder - Decoder Architecture :-



Train Architecture

Dataset:-

Eng	Hindi	or English	Urdu
Think about it	Tik aur	1) Think about it	سوچ لو
		2) come in	اندر آ جاؤ

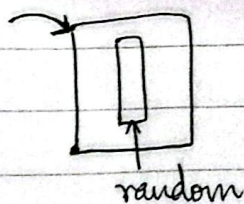
↓
numbers

↓
NLP

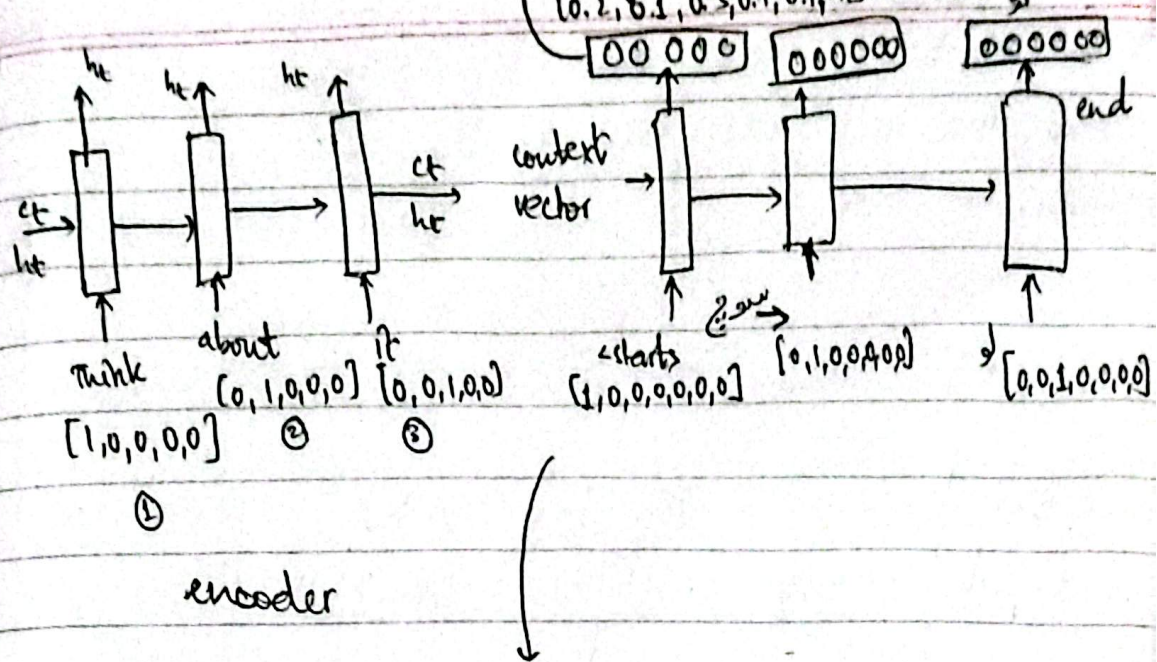
[¹ think, ² about, ³ it]	[³ سوچ, ² لو]
[⁴ come, ⁵ in]	[اندر, ⁴ آ, ⁵ جاؤ]

one hot encoding

eng	urdu
think → [1, 0, 0, 0, 0]	<start>
about → [0, 1, 0, 0, 0]	<end>
it → [0, 0, 1, 0, 0]	[1, 0, 0, 0, 0, 0, 0] ← start
come → [0, 0, 0, 1, 0, 0]	[0, 1, 0, 0, 0, 0, 0] ← سوچ
in → [0, 0, 0, 0, 1, 0]	[0, 0, 1, 0, 0, 0, 0] ← لو
	[0, 0, 0, 1, 0, 0, 0] ← اندر
	[0, 0, 0, 0, 1, 0, 0] ← آ
	[0, 0, 0, 0, 0, 1, 0] ← جاؤ
	[0, 0, 0, 0, 0, 0, 1] ← end



soft-max layer
 # of nodes = # of words in vocabulary
 so softmax generates probability for every word of vocab



here let's say

$$y_{\text{-pred}} = [0.2, 0.1, 0.3, 0.2, 0.1, 0.1, 0]$$

highest probability

under word $\rightarrow$ 'd'

wrong translation,

so, even if, during training our $y_{\text{-pred}}$ is wrong, we provide it right inputs.

↑ Teacher - forcing concept.

softmax layer

↳ $[0, 0, 0, 0, 0, 0, 0]$ 7 neurons

→ softmax layer takes all words of vocabulary & give probabilities. The one with highest probability will be predicted.

words:- start, ϵ , 'd', 'surds', 'T', 'b', end

$$[0.2, 0.1, 0.3, 0.2, 0.1, 0.1, 0]$$

highest $\rightarrow$ model predicts 'd'.

Now we have done forward propagation,
 Now we will use loss function to
 calculate loss.

loss

→ y_{true}	$[0, 1, 0, 0, 0, 0, 0]$	$[0, 0, 1, 0, 0, 0, 0]$	$[0, 0, 0, 0, 0, 0, 1]$
→ $\hat{y}_{\text{pred}}$	$[0.2, 0.1, 0.3, 0.2, 0.1, 0, 0]$	$[0.1, 0.2, 0.1, 0.15, 0.5, 0.5, 0.05]$	$[0.1, 0.2, 0.2, 0.1, 0.1, 0.4]$
	$t=1$	$t=2$	$t=3$

loss → categorical cross entropy

$$L = - \sum_{j=1}^7 y_i^{\text{true}} \log(y_j^{\text{pred}})$$

7 words in
 vocabulary
 right now,
 so 7 categories

$$L_{t=1} = -1 \times \log(0.1) = 1$$

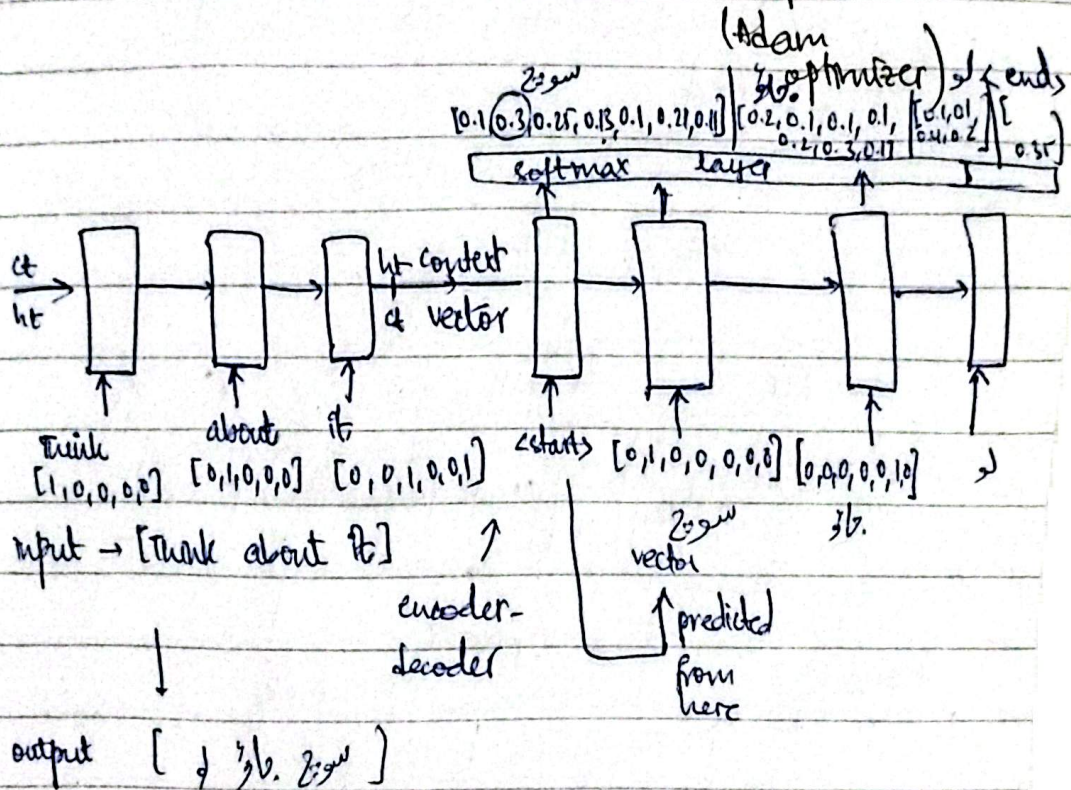
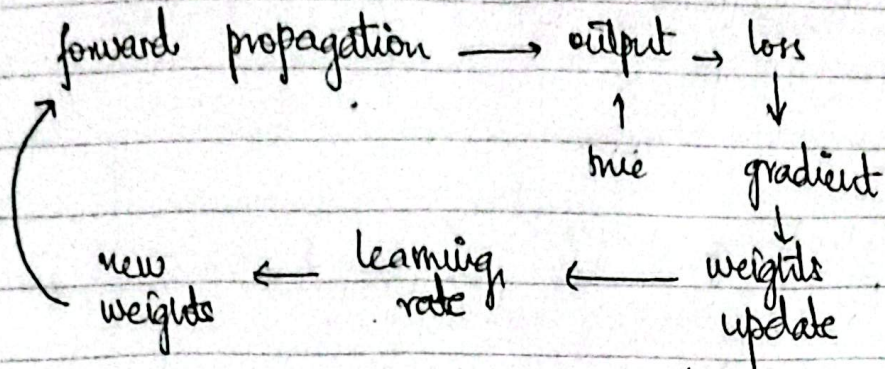
$$L_{t=3} = -1 \times \log(0.4) = 0.39$$

$$L_{\text{total}} = 2.39$$

Now, backpropagation.

① → gradient

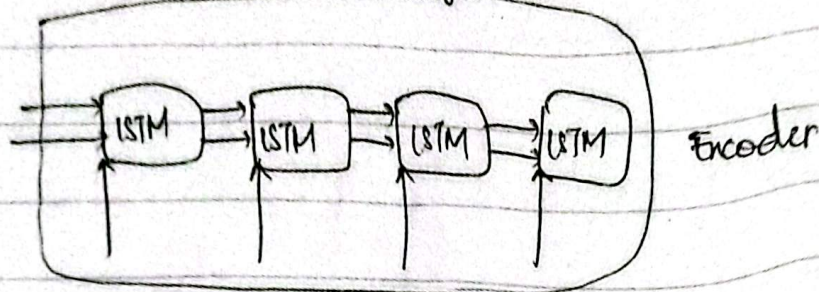
② → update parameters



↑ During testing, we don't do teacher forcing.

Improvement

① Improvement 1 - Embeddings



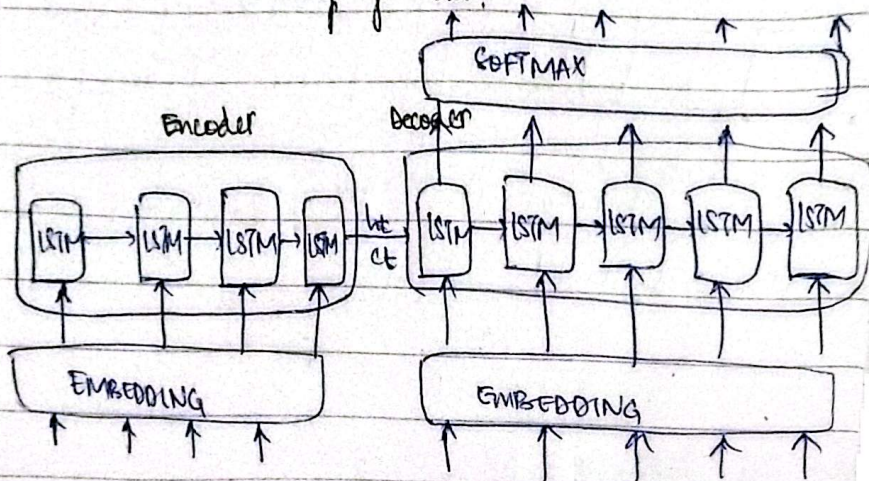
Embedding layer

↳ semantics of word captured

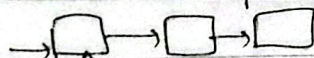
↳ low dimension/dense representation of word

↳ pretrained → Word2Vec

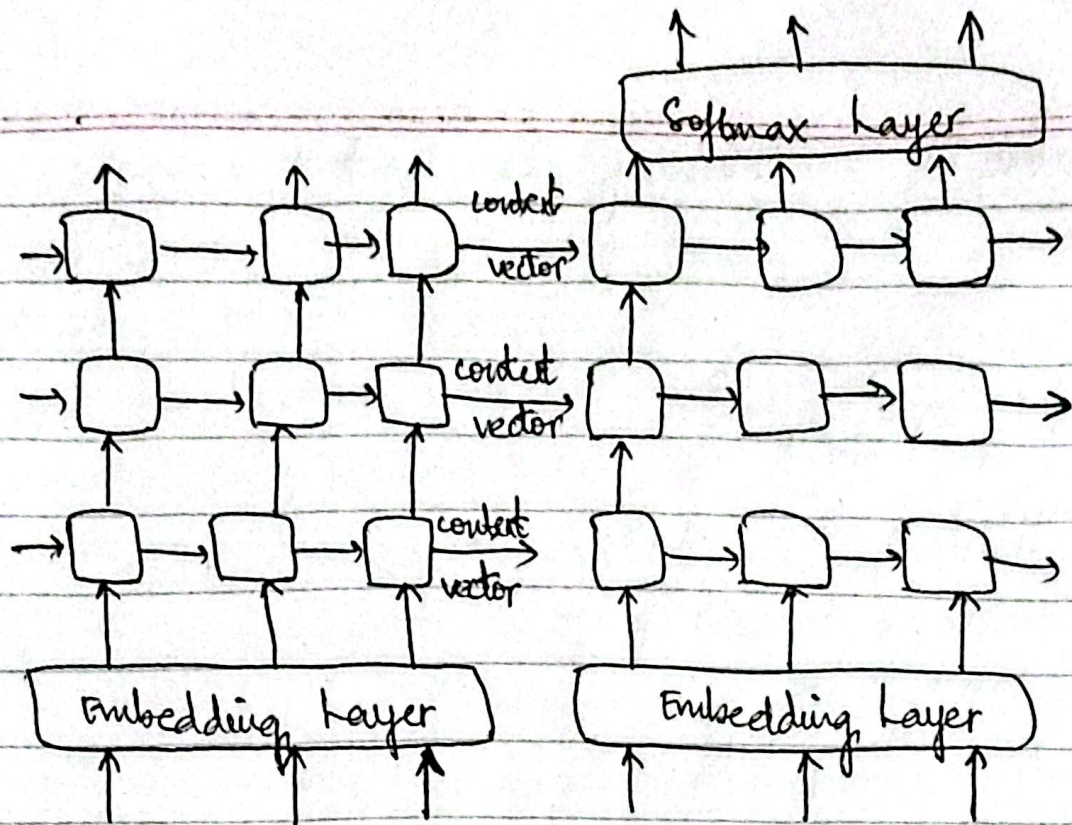
↳ summary of word



Improvement 2 - Deep LSTM



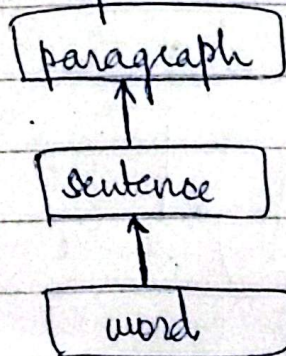
stack hidden layer



Adv.:-

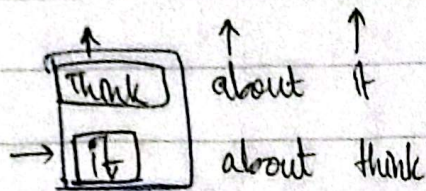
1) long term dependency
 Deep LSTM | every layer of LSTM captures long term dependency → good for large paragraphs.

2) layered representation
 Layered architecture better understands the hierarchical representation of words.



3) Deep learning NN learning
 ↳ learns patterns well.
 ↳ better generalizations well.

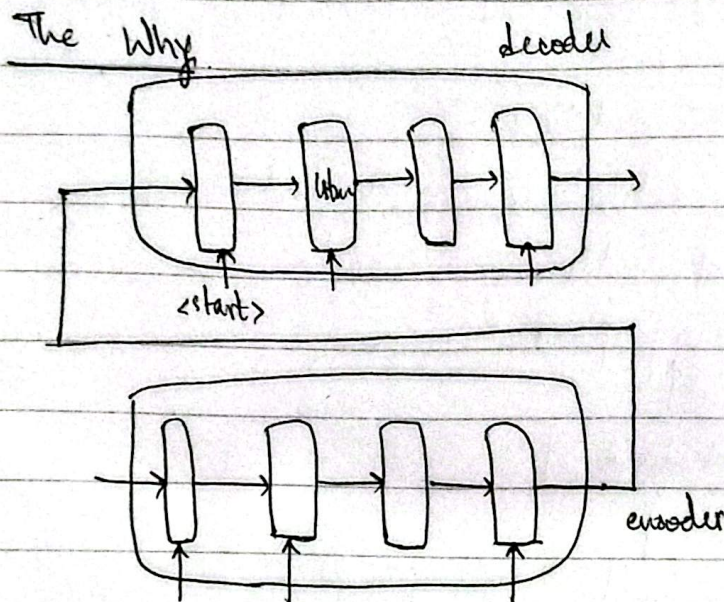
Improvement 3 - Reversing the Input



works for some language pairs only.

works only for language pairs in which initial words hold more context (like English).

ATTENTION MECHANISM :-



Problem :-

long paragraphs → difficult

once upon a time in a small Indian village, a mischievous monkey stole a turban from a sleeping barber, wore it to a wedding, danced with the bewildered guests, accidentally got crowned the 'Banana King' by the local kids, & ended up leading a vibrant, impromptu parade of laughing villagers, cows, & street dogs, all while balancing a stack of mangoes on its head, creating a hilariously unforgettable spectacle & an amusing legend that the village still chuckles about every monsoon season.

↳ very long paragraph

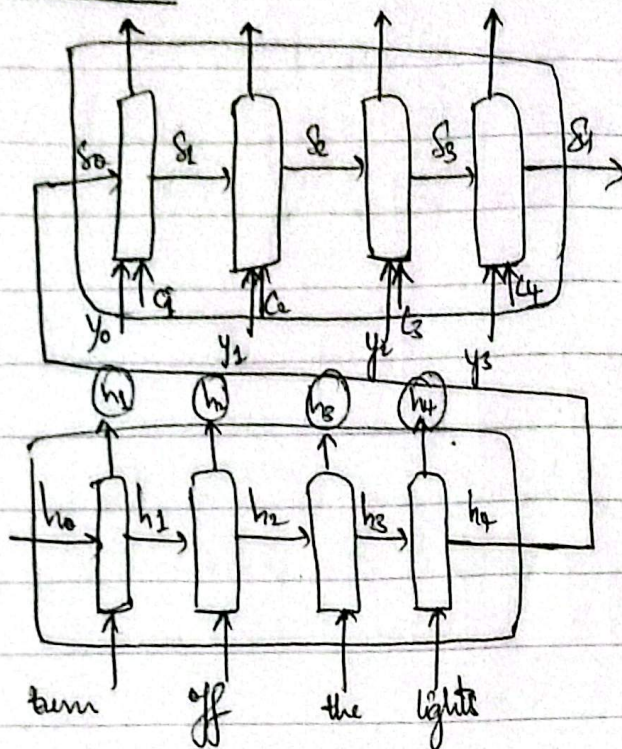
Problem with this Architecture :-

→ For translation of any word, only some words matter not that whole long paragraph. Encoder/Decoder takes that whole long sentence into context.

→ At every time step, we need to see & also translate how & where which timesteps/words are important.

↑
This can be calculated by
Attention Mechanism

The What



$i=2$ prev. in simple encoder decoder
 $t=2$ at Decoder

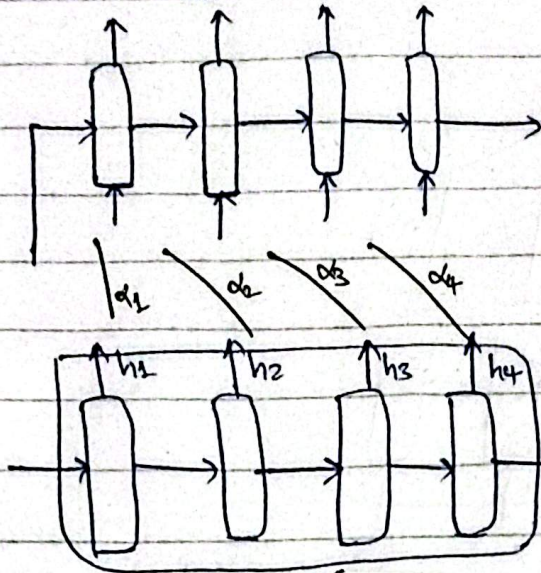
vanilla encoder-decoder i	attention i
$[y_{i-1}, s_{i-1}]$	$[y_{i-1}, s_{i-1}, c_i]$

1) $[c_i] \rightarrow \text{vector} \rightarrow [\text{dim}]$
 $[1 \ 0.5 \ 0.2 \ 0.3]$

$\rightarrow h_2 \rightarrow$ if dimension of h_2 is 4
 $c_1 \rightarrow 4$ dimensions

$\rightarrow h_2, h_3$
 $c_1 \rightarrow$ weighted sum of both

→ $h_2 \ h_3$
weighted
sum



$$c_1 = \alpha_1 h_1 + \alpha_2 h_2 + \alpha_3 h_3 + \alpha_4 h_4$$

vectors

$$c_2 = \alpha_1 h_1 + \alpha_2 h_2 + \alpha_3 h_3 + \alpha_4 h_4$$

α_i will differ at every timestep.

decoder → timestep → call it i

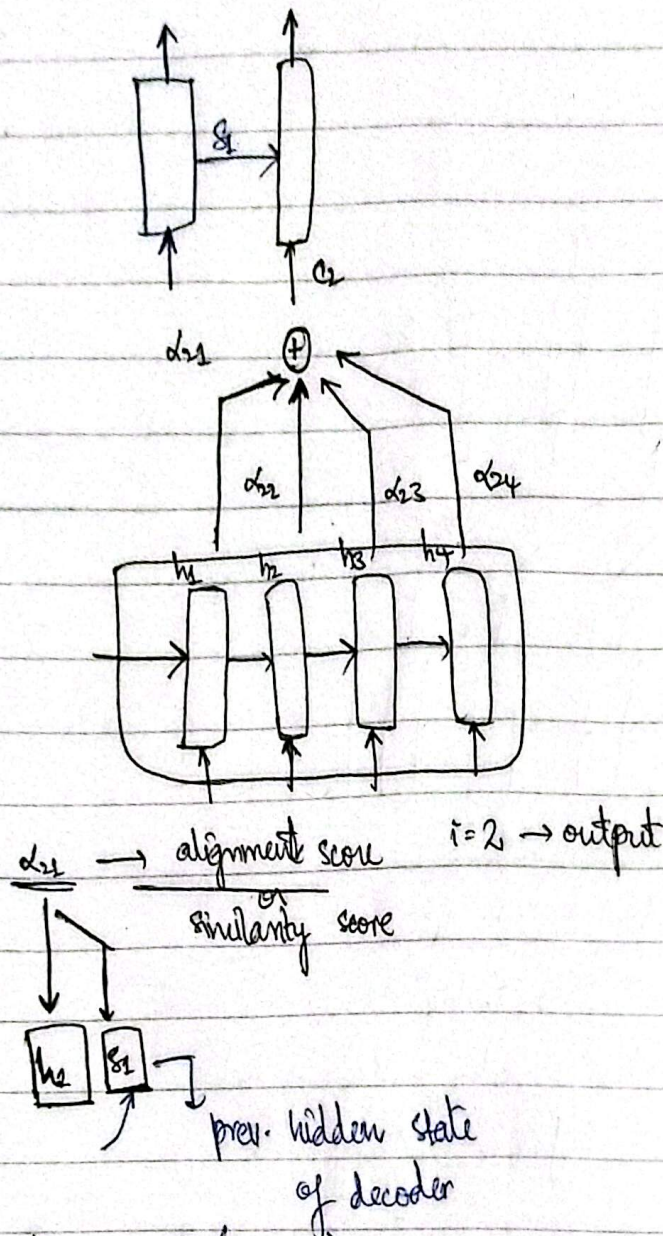
encoder → timestep → call it j .

$$C_i = \sum \alpha_{ij} h_j$$

$$c_2 = \alpha_{21} h_1 + \alpha_{22} h_2 + \alpha_{23} h_3 + \alpha_{24} h_4$$

$$\text{total } \alpha \rightarrow i \times j = 4 \times 4 = 16$$

Now, how to calculate values for α .



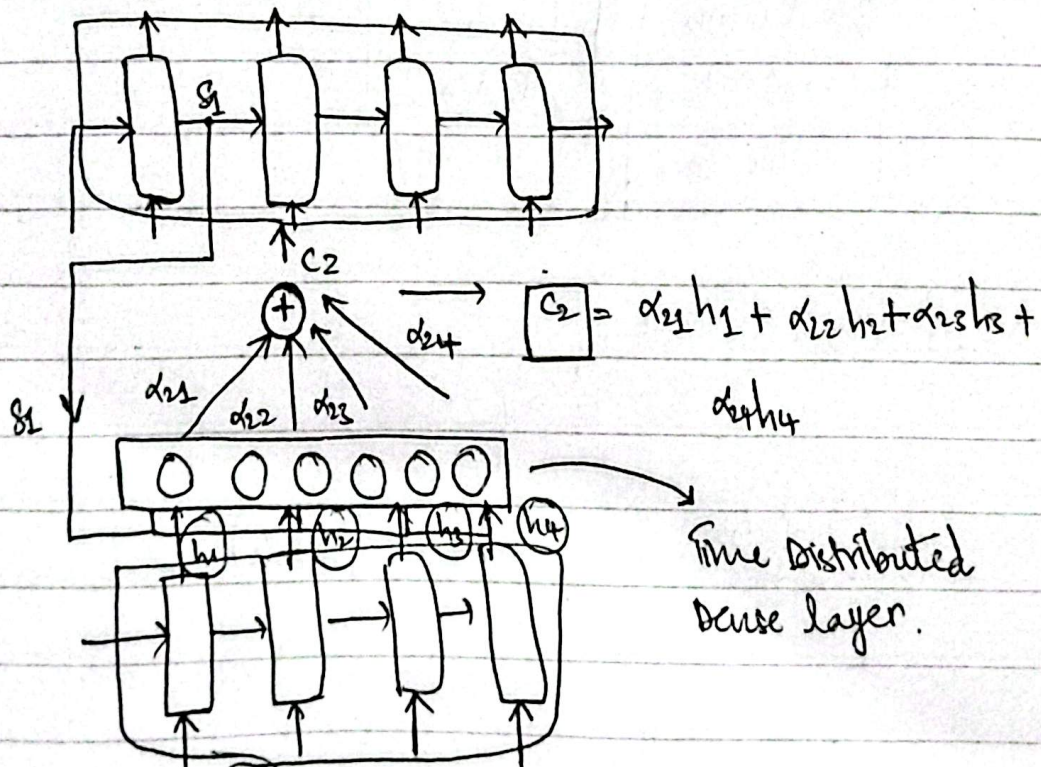
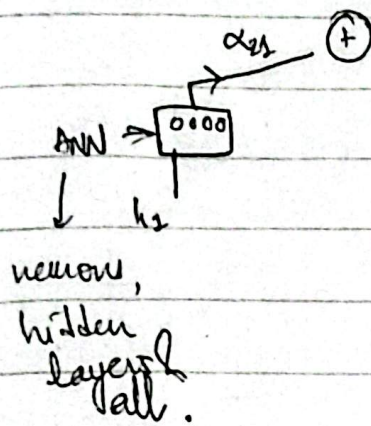
$d_{21} \rightarrow$ alignment score $i=2 \rightarrow$ output
 or
 similarity score
 $d_{21} \rightarrow f(h_2, s_1)$
 $d_{ij} \rightarrow f(h_j, s_{i-1})$
 universal function approximators $\uparrow$ ANN
 what function to use
 hidden state at prev. decoder block
 hidden state output coming from encoder

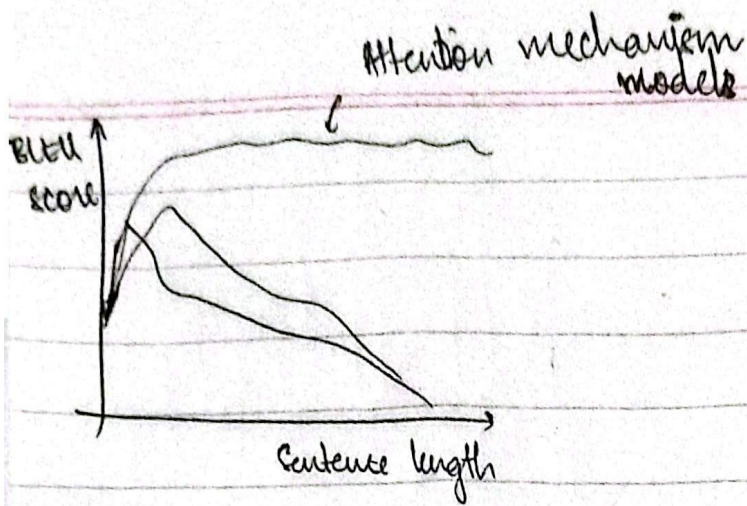
ANN

↑ universal
function
approximator

$$x_{ij} = f(h_j, s_{i-1})$$

ANN (Artificial Neural Network)



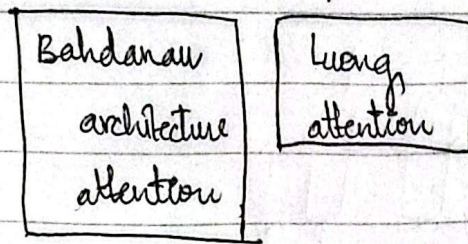


encoder

↳ biLSTM were used.

$$Q = \sum_{i=1}^n \alpha_{ij} h_j$$

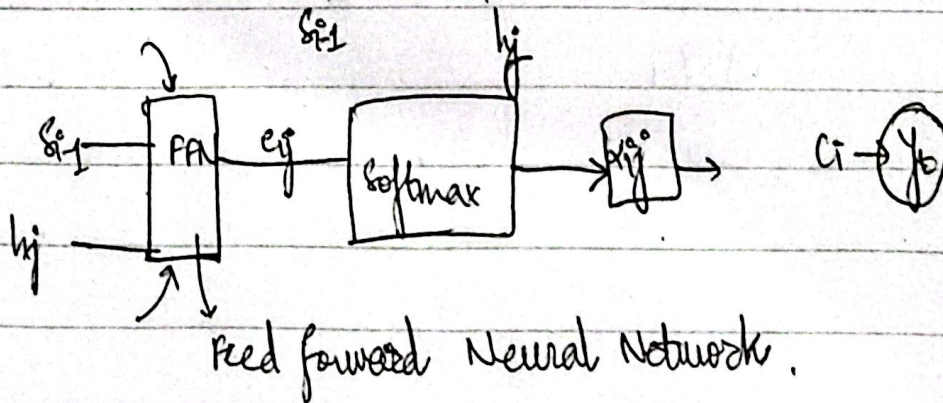
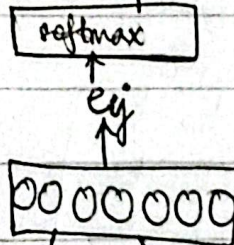
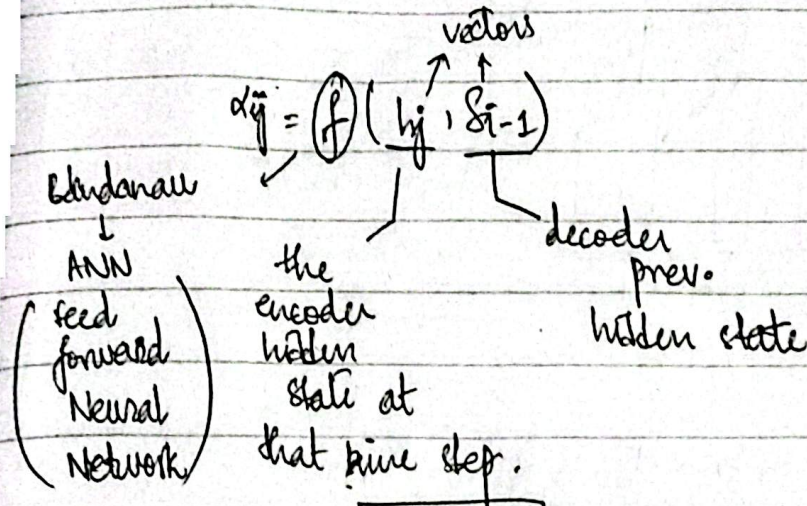
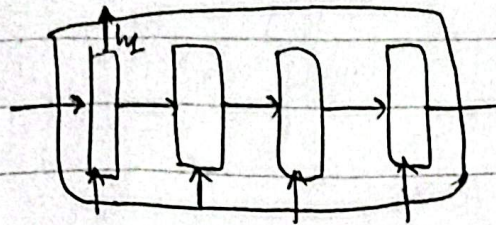
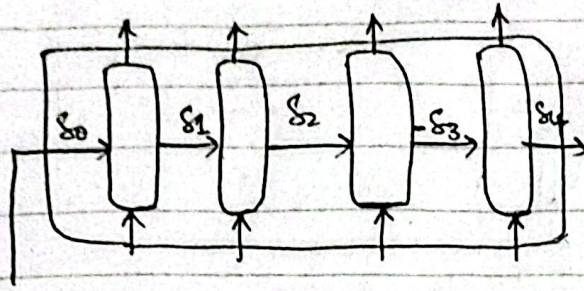
$\alpha \rightarrow$ find out

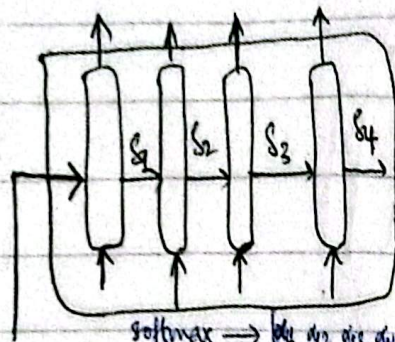


alignment score.

$$Q = \sum \alpha_{ij} \cdot h_j$$

$$\alpha_{11} = f(h_1, s_0) \quad \alpha_{21} = f(h_1, s_1)$$





1) encodes sentences

$\hookrightarrow h_1, h_2, h_3, h_4$

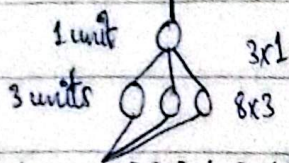
vector 4 dim $s_{t-1} \quad h_j$

$$s_0 = [e \ f \ g \ h]$$

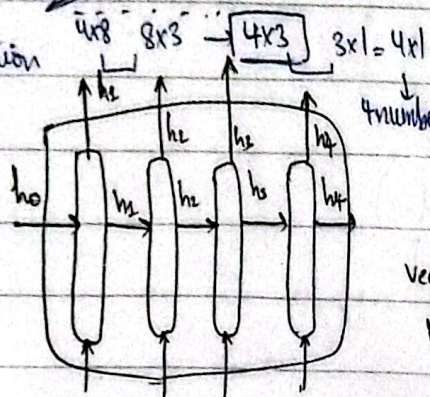
$$c_1 = \sum \alpha_{ij} h_j$$

$$= \alpha_{11} h_1 + \alpha_{12} h_2 + \alpha_{13} h_3 + \alpha_{14} h_4$$

$$\alpha(s_0 = [e \ f \ g \ h])$$



batch operation



vectors

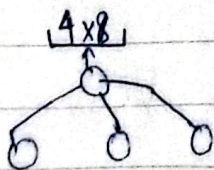
$$h_0 = [a \ b \ c \ d]$$

4 rows				8 cols			
s_{01}	s_{02}	s_{03}	s_{04}	h_{11}	h_{12}	h_{13}	h_{14}
s_{01}	s_{02}	s_{03}	s_{04}	h_{21}	h_{22}	h_{23}	h_{24}
s_{01}	s_{02}	s_{03}	s_{04}	h_{31}	h_{32}	h_{33}	h_{34}
s_{01}	s_{02}	s_{03}	s_{04}	h_{41}	h_{42}	h_{43}	h_{44}

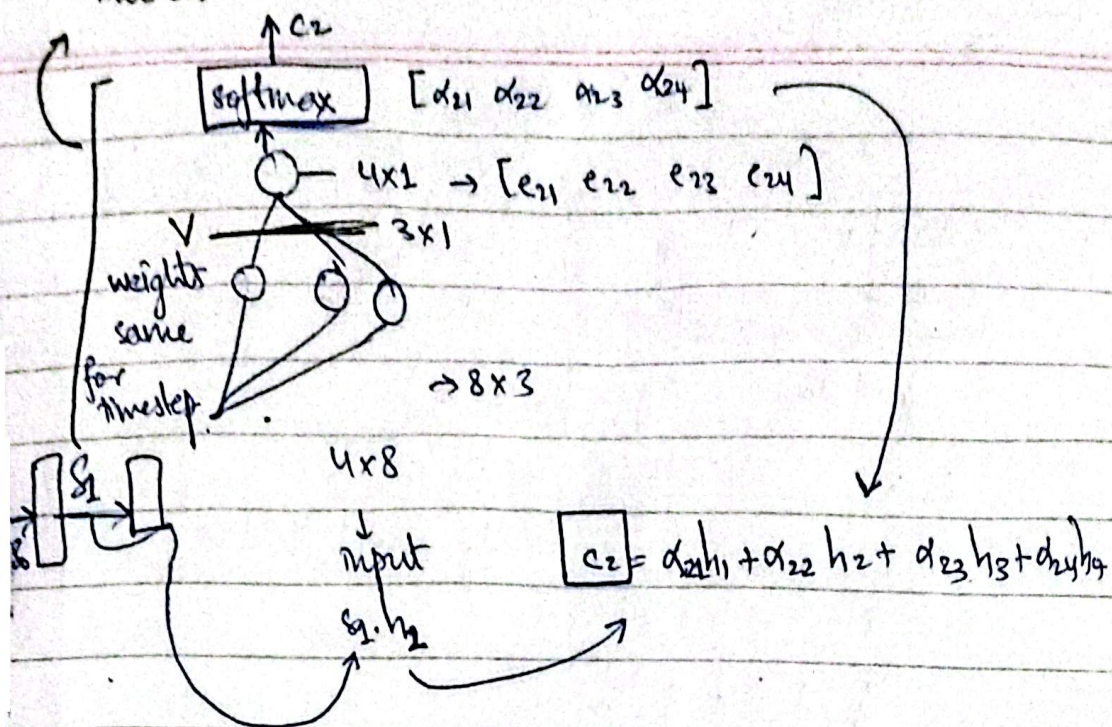
$t=2$

$$c_2 = \alpha_{21} h_1 + \alpha_{22} h_2 + \alpha_{23} h_3 + \alpha_{24} h_4$$

s_{11}	s_{12}	s_{13}	s_{14}	h_{11}	h_{12}	h_{13}	h_{14}
s_{21}	s_{22}	s_{23}	s_{24}	h_{21}	h_{22}	h_{23}	h_{24}
s_{31}	s_{32}	s_{33}	s_{34}	h_{31}	h_{32}	h_{33}	h_{34}
s_{41}	s_{42}	s_{43}	s_{44}	h_{41}	h_{42}	h_{43}	h_{44}



alignment
model



$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^4 \exp(e_{ik})}$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^4 \exp(e_{ik})}$$

$$c_j = \sum_i \alpha_{ij} h_i$$

additive
attention

c_i → context vector given to decoder

Luong Attention

$$c_i = \sum_j \alpha_{ij} h_j$$

FFN (Feed Forward Neural Network)

Bahdanau → $\alpha_{ij} = f(s_{i-1}, h_j) \propto$

It would be more updated current decoder state
↳ dynamic adjustment

Luong Attention → $\alpha_{ij} = f(s_i, h_j)$

softmax
 $s_i^T \cdot h_j \rightarrow$ dot product

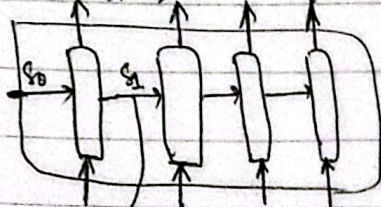
$$s_i = [a \ b \ c \ d]$$

$$h_j = [e \ f \ g \ h]$$

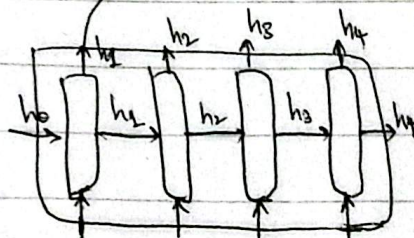
$$\begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} [e \ f \ g \ h]$$

$$\alpha_{ij} \uparrow [ae + bf + cg + dh]$$

softmax ← e_{ij}
concatenate $s_i \parallel e_{ij} \rightarrow \tilde{s}_i \rightarrow$ softmax



softmax
dot product



$s_1 h_1$	$s_1 h_2$	$s_1 h_3$	$s_1 h_4$
e_{11}	e_{12}	e_{13}	e_{14}
↓ softmax	↓ softmax	↓ softmax	↓ softmax
α_{11}	α_{12}	α_{13}	α_{14}